evaluate.py

`scripts/evaluate.py · 275 lines`

What it does

Scores a model stage on the evaluation set: `original`, `before_training` (replaced model) or `after_training` (trained biases). The original model is evaluated first if its results are missing or stale.

Run

```
uv run python scripts/evaluate.py --stage before_training
```

Pipeline order: inspect_model -> build_c4_support_eval -> collect_stats -> score_experts -> replace_experts -> evaluate (before_training) -> train_biases -> evaluate (after_training) -> export_model

Reads

- `EVALUATION_DATASET_PATH`
- the model of the chosen stage
- removed experts' original weights (probe)

Writes

- `evaluation/<stage>_metrics.json`, `<stage>_per_example.pt`
- `evaluation/comparison.json` and `comparison.md`

Notes

- All stages use exactly the same cached examples; only one model is in memory at a time.
- The comparison shows loss change, perplexity (causal models), top-1 agreement, share of examples that got worse, per-layer reconstruction error and qualitative predictions.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`run_stage(stage, cfg, paths, examples, settings, tokenizer, task)`	Load the stage's model, evaluate it and write its metrics.
`write_comparison(paths, settings)`	Combine all evaluated stages into comparison.json / comparison.md.
`main()`	Script entry point: runs this stage end to end.

Settings it reads

`EVAL_BATCH_SIZE`, `EVAL_QUALITATIVE_SAMPLES`, `MODEL_PATH or HF_MODEL_ID`, `REPLACEMENT_METHOD`

Connections

Uses: `moe_pruning.evaluation.evaluator`, `moe_pruning.evaluation.metrics`, `moe_pruning.model.architecture`, `moe_pruning.model.loader`, `moe_pruning.utils.cli`, `moe_pruning.utils.device`, `moe_pruning.utils.paths`, `moe_pruning.utils.reproducibility`